



Departamento de Informática e Matemática Aplicada – DIMAp

Comunicação TCP

Prof. Nélio Cacho

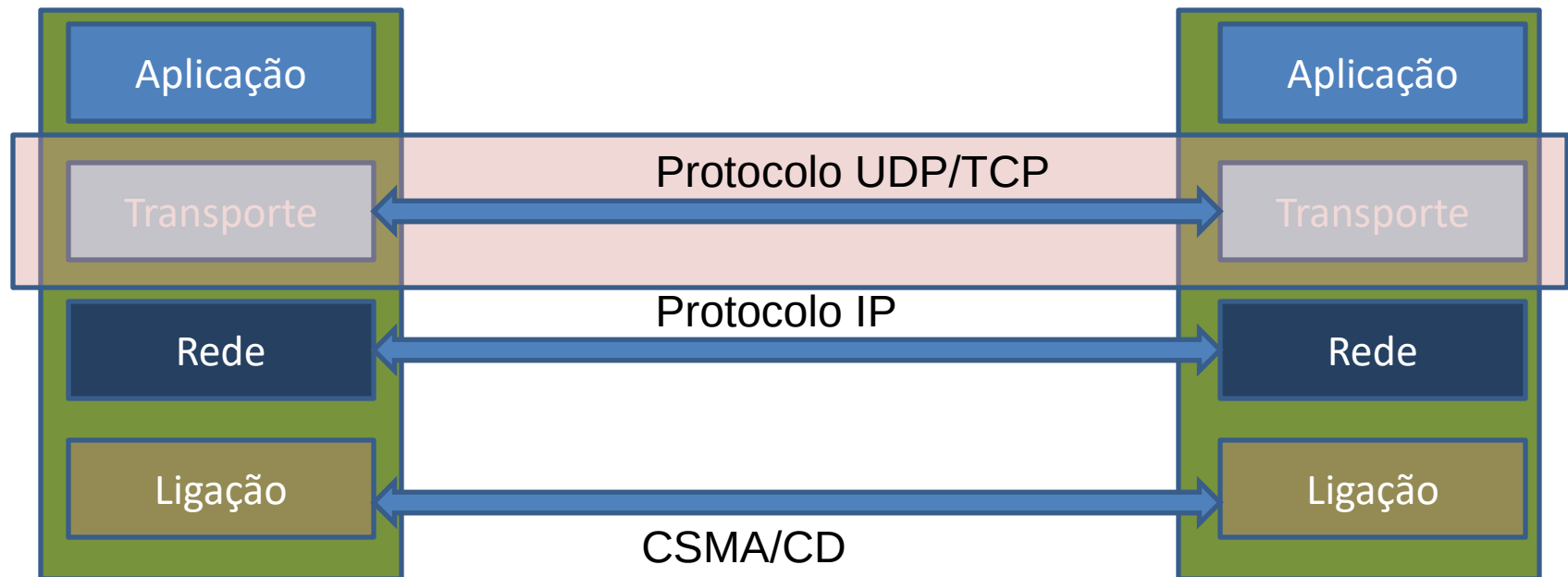
Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.

Como realizar a comunicação ?



Vantagens do TCP

- Controle automático de erro
- Confiabilidade – Dados são entregues na ordem em que são enviados
- Facilidade de Uso: Não existe a necessidade de enviar pacotes de dados separadamente.

Interação TCP Servidor

Servidor começa por criar um TCP Server Socket...

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

```
ServerSocket servSock = new ServerSocket(servPort);
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

```
for (;;) {  
    Socket clntSock = servSock.accept();
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

Servidor fica bloqueado esperando conexões

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente

Um cliente então resolve estabelecer uma comunicação com o servidor...

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente

```
Socket socket = new Socket(server, servPort);
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente

```
OutputStream out = socket.getOutputStream();  
out.write(byteBuffer);
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

```
Socket clntSock = servSock.accept();
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Servidor

```
InputStream in = clntSock.getInputStream();  
recvMsgSize = in.read(byteBuffer);
```

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Interação TCP Cliente/Servidor

`close(sock);`

`close(clntSocket)`

Cliente

1. Cria um socket TCP
2. Comunica
3. Fecha conexão

Servidor

1. Cria um socket TCP
2. Repetidamente:
 - a. Aceita novas conexões
 - b. Comunica
 - c. Fecha a conexão

Exemplo Cliente TCP Enviando Objetos

```
public class TCPClient {  
    public static void main(String args[]) {  
        Socket conexao = null;  
        ObjectOutputStream output = null;  
        ObjectInputStream input = null;  
        try {  
            System.out.println("TCP Client Bank Started");  
            conexao = new Socket("localhost", 8080);  
            output = new ObjectOutputStream(conexao.getOutputStream());  
            output.writeObject("criar;1;1000");  
            output.flush();  
            input = new ObjectInputStream(conexao.getInputStream());  
            String msg = (String) input.readObject();  
            System.out.println("Retorno do Servidor:"+msg);  
        } catch (UnknownHostException e) {  
            e.printStackTrace();  
        } catch (IOException e) {  
            e.printStackTrace();  
        } catch (ClassNotFoundException e) {  
            e.printStackTrace();  
        } finally {  
            try {  
                input.close();  
                output.close();  
                conexao.close();  
            } catch (IOException e) {  
                e.printStackTrace();  
            }  
        }  
        System.out.println("TCP Client Bank Finished");  
    }  
}
```

Exemplo Servidor TCP Recebendo Objetos

```
public class TCPServer {  
    public static void main(String args[]) {  
        System.out.println("TCP Server Bank Started");  
        ServerSocket server = null;  
        Banco banco = new Banco();  
        ProcessPayload processpayload = new ProcessPayload(banco);  
        try {  
            server = new ServerSocket(8080);  
        } catch (IOException e2) {  
            e2.printStackTrace();  
        }  
        while (true) {  
            try {  
                Socket conexao = server.accept();  
                ObjectInputStream input = new ObjectInputStream(conexao.getInputStream());  
                String msg = (String) input.readObject();  
                System.out.println("Operação recebida:"+msg);  
  
                String reply = processpayload.processData(msg,conexao.getInetAddress().toString());  
  
                ObjectOutputStream output = new ObjectOutputStream(conexao.getOutputStream());  
                output.writeObject(reply);  
                output.flush();  
                output.close();  
                conexao.close();  
            } catch (IOException e) {  
                e.printStackTrace();  
            } catch (ClassNotFoundException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }  
        }  
    }  
}
```


Exemplo Servidor TCP Recebendo Objetos

```
public class ProcessPayload {
    private Banco banco;
    public ProcessPayload(Banco bank) {
        this.banco = bank;
    }
    public String processData(String msg, String Ip) {
        String operacao = null;
        int conta = 0;
        int valor = 0;

        StringTokenizer tokenizer = new StringTokenizer(msg, ";");
        while (tokenizer.hasMoreElements()) {
            operacao = tokenizer.nextToken();
            conta = Integer.parseInt(tokenizer.nextToken());
            valor = Integer.parseInt(tokenizer.nextToken().trim());
        }
        String reply = "Operacao realizada:" + operacao + "-" + Ip;
        String resultadoOp = reply;
        switch (operacao) {
            case "criar":
                banco.addConta(conta);
                break;
            case "depositar":
                banco.depositar(conta, valor);
                break;
            case "saldo":
                resultadoOp = "Saldo é R$"+banco.saldo(conta);
                break;
        }

        System.out.println(resultadoOp);
        return resultadoOp;
    }
}
```

Testar Serviço TCP com JMeter

- Para criar um Sampler TCP, clicar com botão direito no ThreadGroup, "add"-> "Sampler"-> "TCP Sampler".

The image shows the configuration window for a TCP Sampler in Apache JMeter. The window has a dark theme. At the top, it says "TCP Sampler". Below this, there are several input fields and checkboxes. The "Name" field contains "Criar Conta". The "Comments" field is empty. The "TCPClient classname" field contains "TCPClientImpl". The "Target Server" section includes "Server Name or IP" (127.0.0.1), "Port Number" (8080), "Connect" (empty), and "Response" (empty). There are checkboxes for "Re-use connection", "Close connection", and "Set NoDelay", all of which are unchecked. Below these are fields for "SO_LINGER" and "End of line(EOL) byte value", both empty. A "Text to send" section contains a list box with two items: "1" and "2". The text "criar;1;1000" is entered in the text area next to item "1". At the bottom, there is a "Login Configuration" section with "Username" and "Password" fields, both empty.

TCP Sampler

Name:

Comments:

TCPClient classname:

Target Server

Server Name or IP: Port Number: Connect: Response:

Timeouts (milliseconds)

Re-use connection ☐ Close connection ☐ Set NoDelay ☐ SO_LINGER: End of line(EOL) byte value:

Text to send

1	criar;1;1000
2	

Login Configuration

Username:

Password:

Criar Sampler TCP

TCP Sampler

Name:

Comments:

TCPClient classname:

Target Server

Server Name or IP: Port Number:

Re-use connection ☐ Close connection ☐ Set NoDelay ☐ SO_LINGER: End of line(EOL) byte value:

Text to send

```
1 criar;1;1000
2
```

Login Configuration

Username:

Password:

Endereço e Porta que o servidor TCP foi iniciado

Colocar dado a ser enviado. Lembrar de colocar linha em branco (\n)

Criar Sampler TCP

TCP Sampler

Name:

Comments:

TCPClient classname:

Target Server

Server Name or IP: Port Number:

Re-use connection ☐ Close connection ☐ Set NoDelay ☐ SO_LINGER: End of line(EOL) byte value:

Text to send

```
1 criar;1;1000
2
```

Login Configuration

Username:

Password:

Tipo de Handler:

- TCPClientImpl (padrão)
- BinaryTCPClientImpl
- LengthPrefixedBinaryTCPClientImpl

Criar Sampler TCP

- Um novo Handler "must extend":
 - **org.apache.jmeter.protocol.tcp.sampler.TCPClient**

TCPClientImpl

This implementation is fairly basic. When reading the response, it reads until the end of line byte, if this is defined by setting the property **tcp.eolByte**, otherwise until the end of the input stream. You can control charset encoding by setting **tcp.charset**, which will default to Platform default encoding.

BinaryTCPClientImpl

This implementation converts the GUI input, which must be a hex-encoded string, into binary, and performs the reverse when reading the response. When reading the response, it reads until the end of message byte, if this is defined by setting the property **tcp.BinaryTCPClient.eomByte**, otherwise until the end of the input stream.

LengthPrefixedBinaryTCPClientImpl

This implementation extends **BinaryTCPClientImpl** by prefixing the binary message data with a binary length byte. The length prefix defaults to 2 bytes. This can be changed by setting the property **tcp.binarylength.prefix.length**.

Executando Casos de Teste

TCP Sampler

Name:

Comments:

TCPClient classname:

Target Server

Server Name or IP: Port Number: Connect: Response:

Timeouts (milliseconds)

Re-use connection ☐ Close connection ☐ Set NoDelay ☐ SO_LINGER: End of line(EOL) byte value:

Text to send

1 criar;1;1000
2

Login Configuration

Username:

Password:

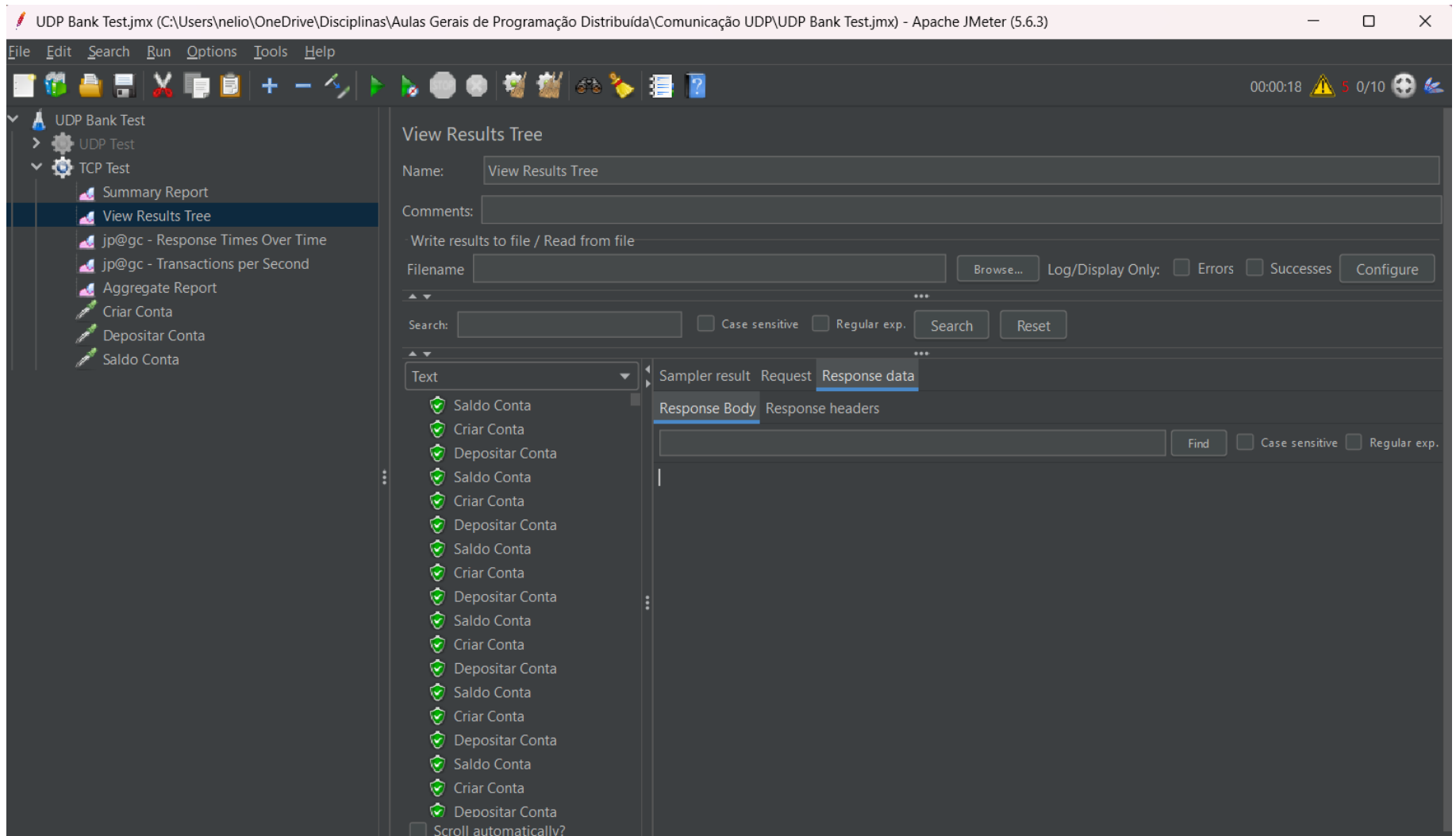
Servidor espera um objeto

```
TCP Server Bank Started
java.io.StreamCorruptedException: invalid stream header: 63726961
    at java.base/java.io.ObjectInputStream.readStreamHeader(ObjectInputStream.java:916)
    at java.base/java.io.ObjectInputStream.<init>(ObjectInputStream.java:386)
    at ProtocoloTCP/ufrrn.imd.br.TCPServer.main(TCPServer.java:30)
java.io.StreamCorruptedException: invalid stream header: 63726961
    at java.base/java.io.ObjectInputStream.readStreamHeader(ObjectInputStream.java:916)
    at java.base/java.io.ObjectInputStream.<init>(ObjectInputStream.java:386)
    at ProtocoloTCP/ufrrn.imd.br.TCPServer.main(TCPServer.java:30)
java.io.StreamCorruptedException: invalid stream header: 63726961
    at java.base/java.io.ObjectInputStream.readStreamHeader(ObjectInputStream.java:916)
    at java.base/java.io.ObjectInputStream.<init>(ObjectInputStream.java:386)
    at ProtocoloTCP/ufrrn.imd.br.TCPServer.main(TCPServer.java:30)
java.io.StreamCorruptedException: invalid stream header: 63726961
    at java.base/java.io.ObjectInputStream.readStreamHeader(ObjectInputStream.java:916)
    at java.base/java.io.ObjectInputStream.<init>(ObjectInputStream.java:386)
    at ProtocoloTCP/ufrrn.imd.br.TCPServer.main(TCPServer.java:30)
```

Nova versão do Servidor

```
public class TCPServerRaw {  
    public static void main(String args[]) {  
        System.out.println("TCP Server Bank Started");  
        ServerSocket server = null;  
        Banco banco = new Banco();  
        ProcessPayload processpayload = new ProcessPayload(banco);  
        try {  
            server = new ServerSocket(8080);  
        } catch (IOException e2) {  
            e2.printStackTrace();  
        }  
        while (true) {  
            try {  
                Socket conexao = server.accept();  
                BufferedReader input = new BufferedReader(new InputStreamReader(conexao.getInputStream()));  
                String msg = input.readLine();  
                System.out.println("Operação recebida:" + msg);  
  
                String reply = processpayload.processData(msg, conexao.getInetAddress().toString());  
  
                PrintWriter output = new PrintWriter(conexao.getOutputStream(), true);  
                output.println("Server response: " + reply);  
                output.flush();  
                conexao.close();  
            } catch (IOException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

Agora deu certo!!!



Vamos agora rodar os testes

jp@gc - Ultimate Thread Group

Name:

TCP Test

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

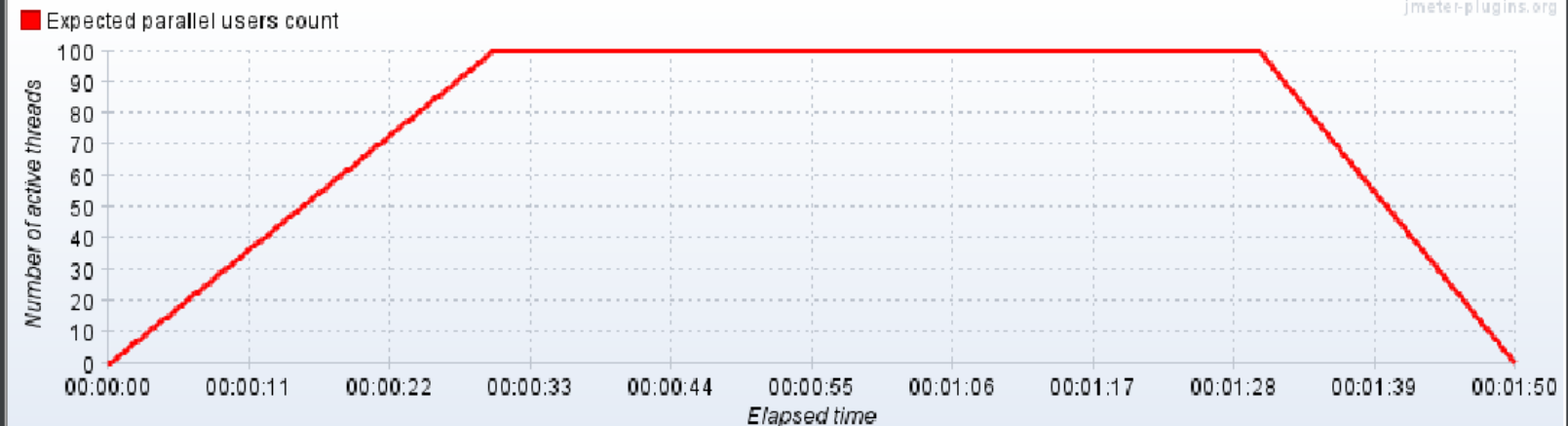
Threads Schedule

Start Threads Count	Initial Delay, sec	Startup Time, sec	Hold Load For, sec	Shutdown Time
100	0	30	60	20

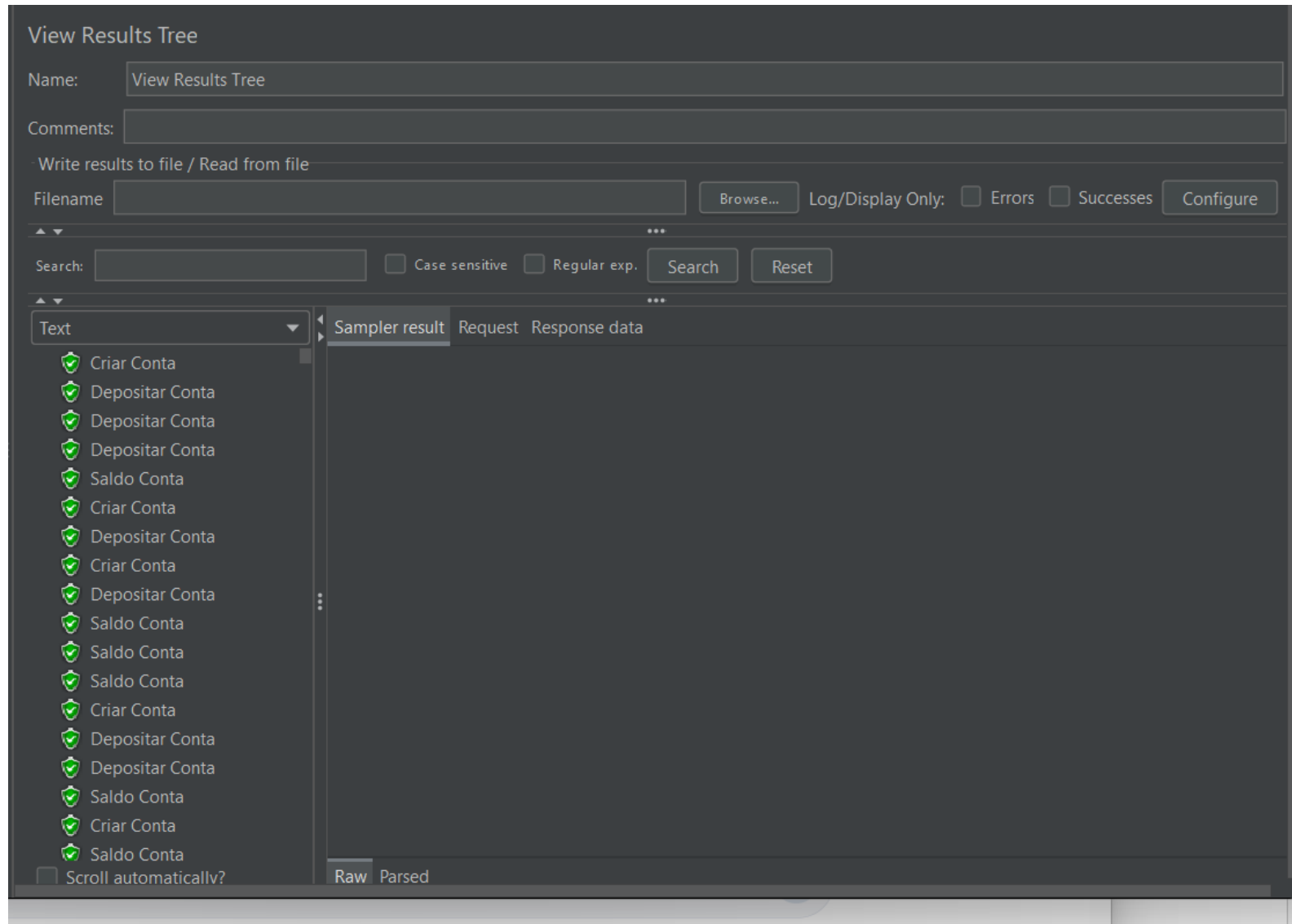
Add Row

Copy Row

Delete Row



Vai tudo indo bem... Até...



Vários erros aparecerem do nadaaa....



View Results Tree

Name: View Results Tree

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes

Search: Case sensitive Regular exp. Search Reset

Text

- ✖ Saldo Conta
- ✖ Criar Conta
- ✖ Saldo Conta
- ✔ Criar Conta
- ✖ Criar Conta
- ✔ Criar Conta
- ✖ Criar Conta
- ✖ Criar Conta
- ✖ Depositar Conta
- ✔ Criar Conta
- ✖ Depositar Conta
- ✖ Criar Conta
- ✖ Saldo Conta
- ✖ Criar Conta
- ✖ Criar Conta
- ✖ Criar Conta
- ✖ Depositar Conta
- ✖ Criar Conta

Sampler result Request Response data

Thread Name:TCP Test 1-13
Sample Start:2025-03-26 18:37:40 GMT-03:00
Load time:2
Connect Time:2
Latency:0
Size in bytes:0
Sent bytes:0
Headers size in bytes:0
Body size in bytes:0
Sample Count:1
Error Count:1
Data type ("text"|"bin"|""): Response code:500
Response message:java.net.ConnectException: Connection refused: connect

SampleResult fields:
ContentType:
DataEncoding: null

Raw Parsed

Scroll automatically?

É preciso mudar a configuração padrão

```
try {  
    server = new ServerSocket(8080);  
} catch (IOException e2) {  
    e2.printStackTrace();  
}
```

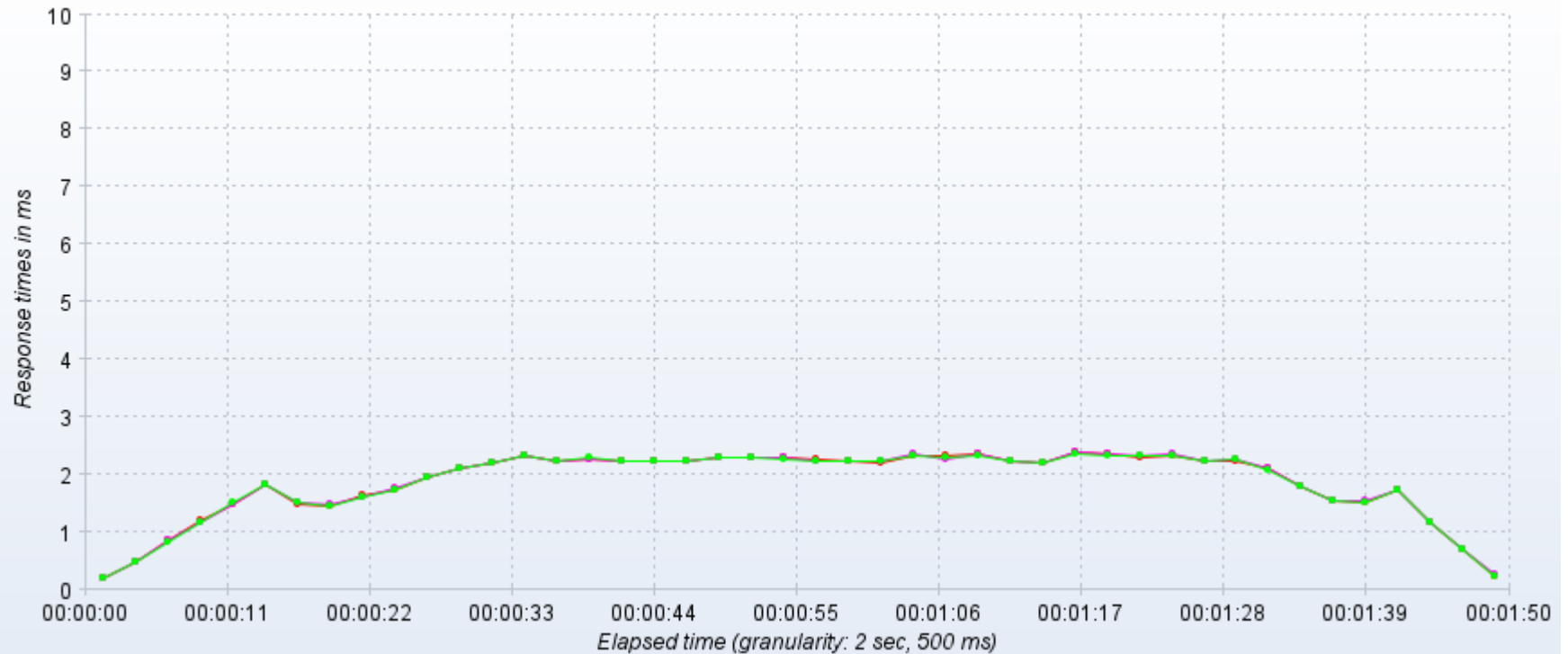


```
try {  
    server = new ServerSocket(8080, 300);  
} catch (IOException e2) {  
    e2.printStackTrace();  
}
```

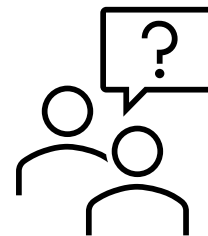
Vamos investigar....

■ Criar Conta ■ Depositar Conta ■ Saldo Conta

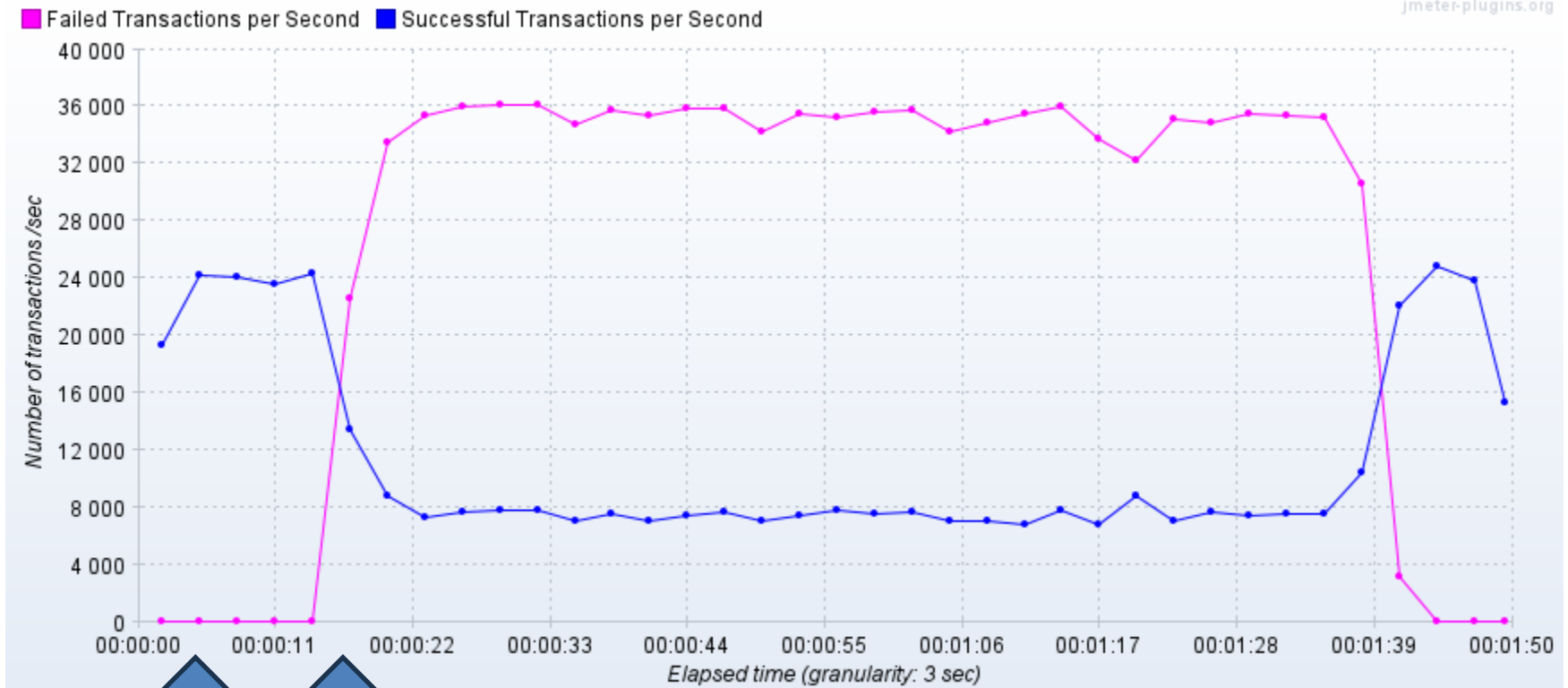
jmeter-plugins.org



Response time parece normal....

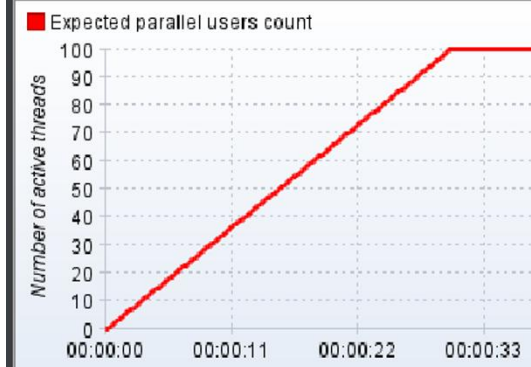
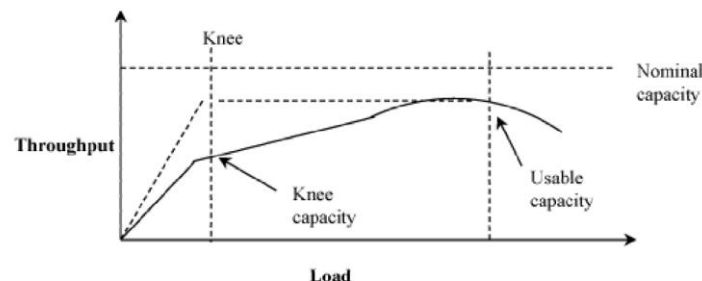


Vamos investigar....quantos usuários meu sistema suporta?



knee

Usable



Minha solução esta com sérios problemas de escalabilidade se eu precisar servir mais de 40 usuários!!!!

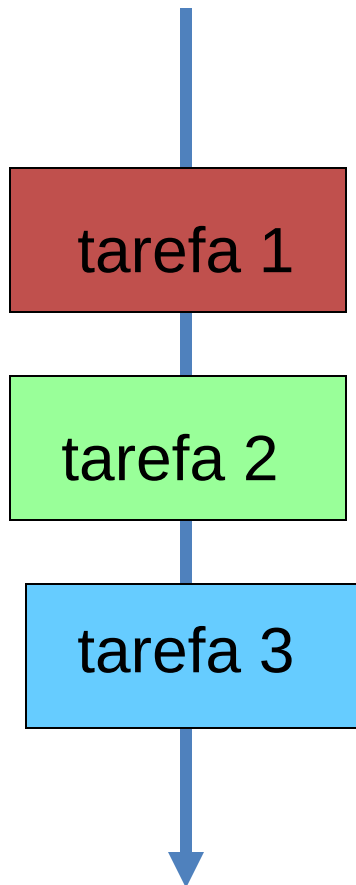
[illegible]

Network Scalability

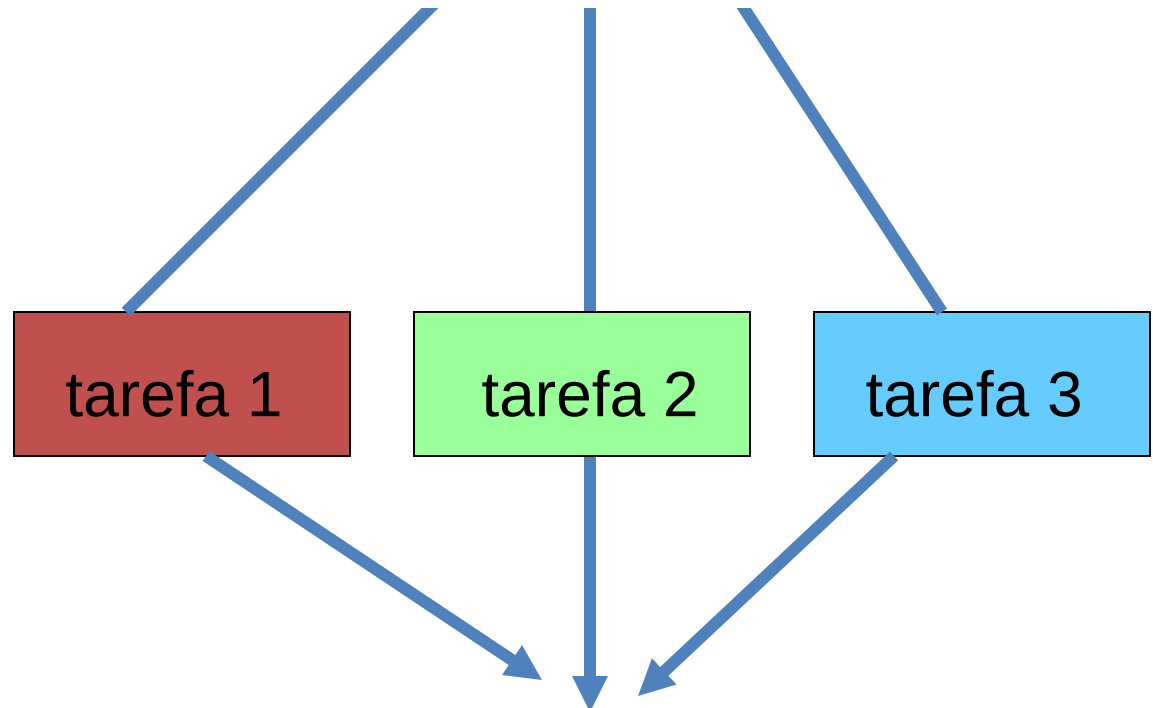
- Server capacity (more servers)
- Multiple threads
- Network bandwidth and latency
- Execution environment (more memory)

Fluxo Sequencial x Concorrente

Fluxo único de execução



Vários fluxos de execução



cada fluxo possui uma pilha de execução

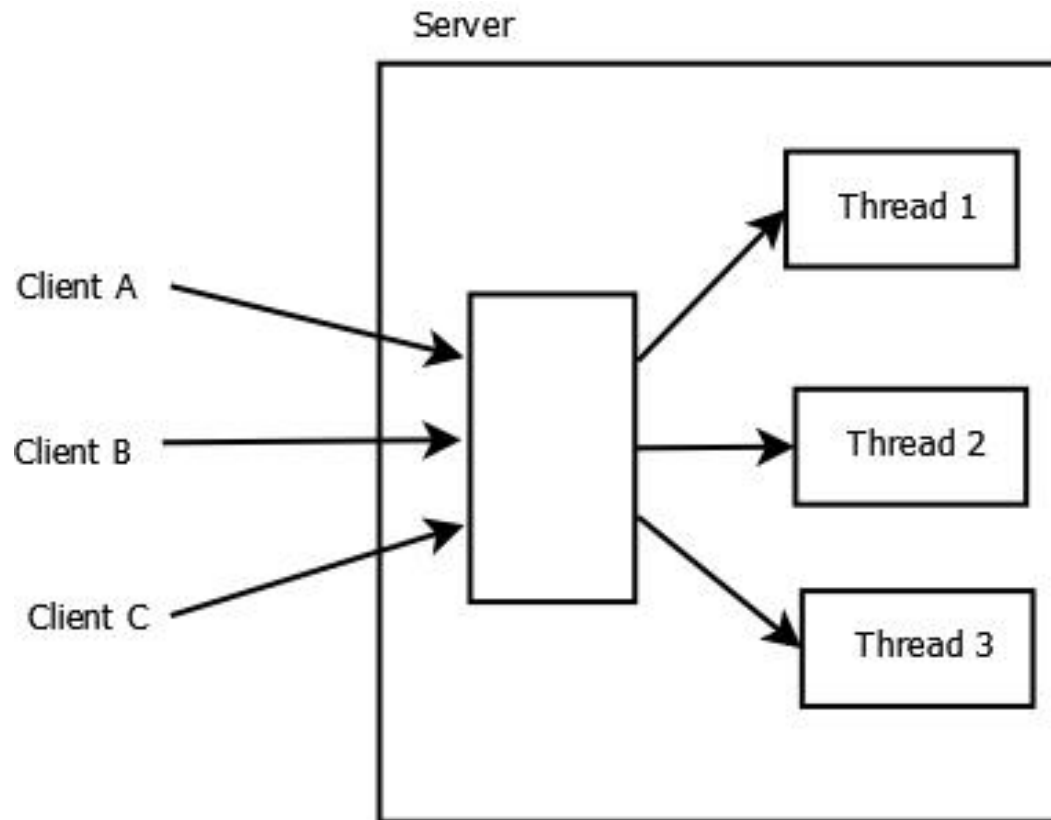
Threads X Processos

- Threads
 - threads dentro de um mesmo processo compartilham o mesmo espaço de endereçamento
 - comunicação entre threads de um mesmo processo: memória compartilhada
- Processos:
 - nada compartilham entre si.
 - tempo de chaveamento de contexto entre processos é alto
 - espaço de endereçamento protegido => comunicação entre processos: socket, pipes

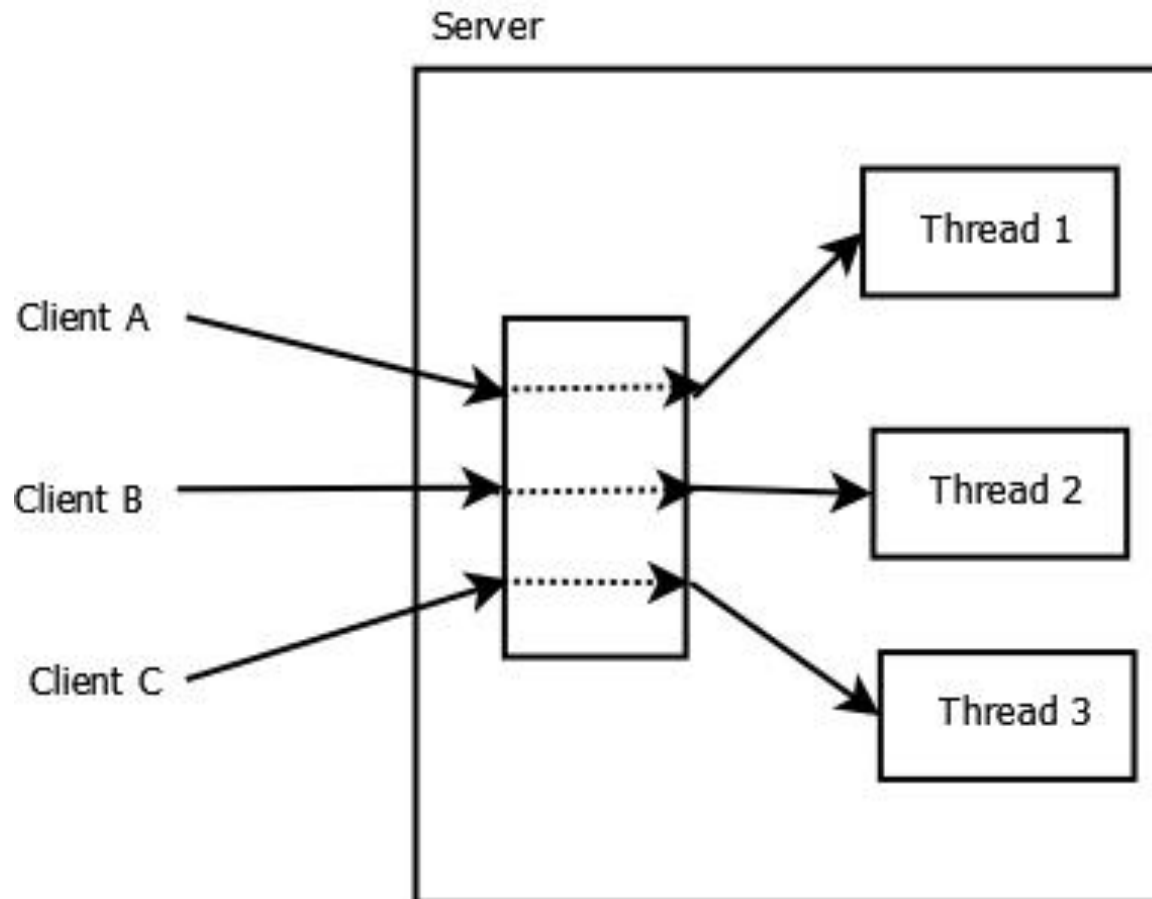
Multithreaded server overview

- When using a multithreaded server, there are several of ways of configuring the threads as follows:
 - Thread-per-request
 - Thread-per-connection
 - Thread-per-object

Thread-per-request model



Thread-per-connection model



Thread-per-object approach

- The thread-per-object approach queues associated requests with a specific object that can handle the request. The object and its methods are placed in a thread that handles requests one at a time. Requests are queued with the thread. While we will not demonstrate this approach here, it is often used with thread pools.
- The process of creating and deleting connections can be expensive. If a client submits several requests, then opening and closing a connection becomes expensive and should be avoided.

Threads em JAVA

- Existe duas formas de criar Threads em Java:
 - Por extensão da classe Thread
 - Por implementação da interface **Runnable**

Criando Threads (Herança)

```
public class SimpleThread extends Thread {  
    public SimpleThread(String str) {  
        super(str);  
    }  
    public void run() {  
        [ código do thread ]  
    }  
}
```


A interface Runnable

- Threads criadas por meio da implementação da interface Runnable são criadas quando uma determinada classe não pode estender a classe Thread (**Java não possui herança múltipla**).
- A solução encontrada em Java foi a utilização de uma interface: Runnable
public class Filho extends Pai implements Runnable

Thread `uma_Thread` = `new` Thread(`Runnable obj_thread`)

Implementação multi-thread

```
public class TCPServerRawNThreadPlatform {
    private static final int THREAD_POOL_SIZE = 100;
    private static final ExecutorService threadPool = Executors.newFixedThreadPool(THREAD_POOL_SIZE);

    public static void main(String args[]) {
        System.out.println("TCP Server Bank Started");
        Banco banco = new Banco();
        ProcessPayload processPayload = new ProcessPayload(banco);

        try (ServerSocket server = new ServerSocket(8080)) {
            while (true) {
                try {
                    Socket conexao = server.accept();
                    threadPool.execute(() -> handleClient(conexao, processPayload));
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            threadPool.shutdown();
        }
    }

    private static void handleClient(Socket conexao, ProcessPayload processPayload) {
        try (conexao;
            BufferedReader input = new BufferedReader(new InputStreamReader(conexao.getInputStream()));
            PrintWriter output = new PrintWriter(conexao.getOutputStream(), true)) {

            String msg = input.readLine();
            System.out.println("Operação recebida de " + conexao.getInetAddress() + ": " + msg);

            String reply = processPayload.processData(msg, conexao.getInetAddress().toString());
            output.println("Server response: " + reply);

        } catch (IOException e) {
            System.err.println("Error handling client: " + e.getMessage());
        }
    }
}
```